hashlock.

Security Audit

Hyswap (DeFi)

March 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

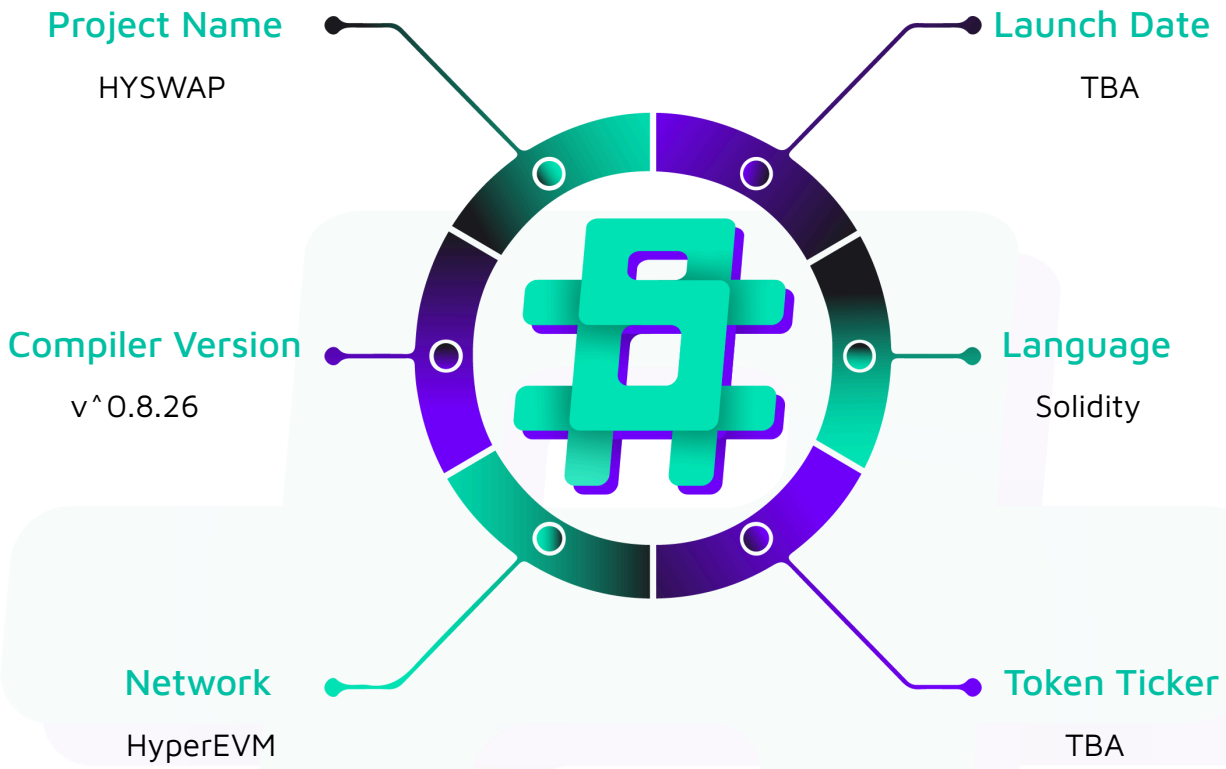
The Hyswap team partnered with Hashlock to conduct a security audit of their smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

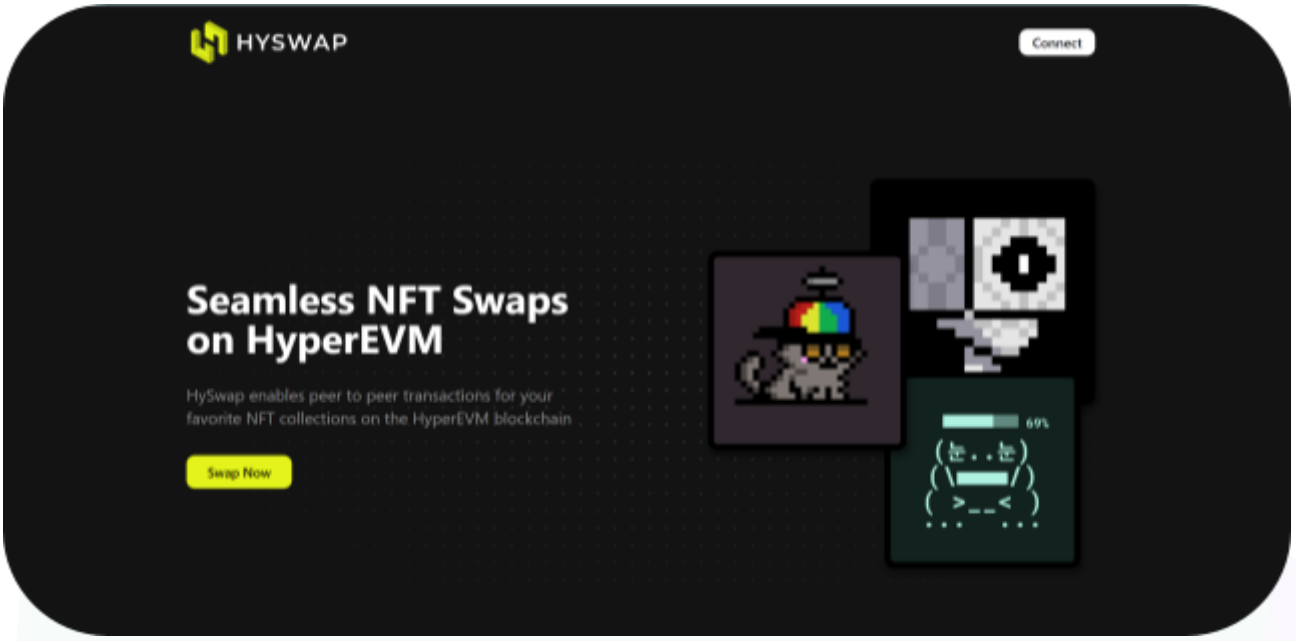
Hyswap is a decentralized platform designed to enable seamless peer-to-peer NFT trading on the HyperEVM blockchain. It allows users to securely and efficiently swap NFTs from different collections, offering a smooth and transparent marketplace experience. With a focus on providing a user-friendly interface and a decentralized environment, Hyswap aims to enhance the accessibility and liquidity of NFTs, making it easier for collectors and traders to engage in NFT transactions without relying on traditional, centralized platforms.

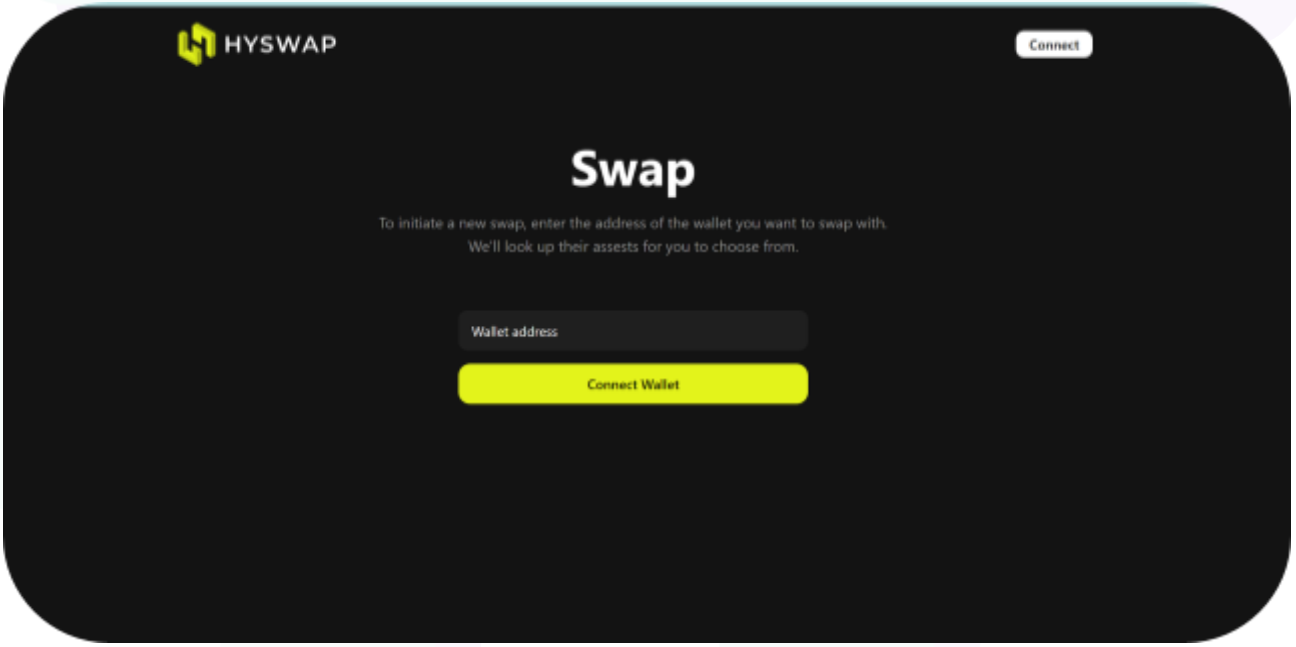
Project Name: Hyswap
Project Type: DeFi
Compiler Version: ^0.8.26
Website: www.hyswap.io
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Hyswap project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Hyswap Smart Contract**
Platform	**EVM / Solidity**
Audit Date	**March, 2025**
Contract	HySwap.sol
Contract MD5 Hash	fcc045456a81b00b620f598e62c920ce

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

2 Medium severity vulnerabilities

1 Low severity vulnerability

2 Gas Optimisations

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

HySwap.sol
- Allows users to:
  - Create a swap offer
  - Accept a swap offer
  - Cancel a swap offer
- Allows owner to:
  - Pause/Unpause contract
  - Verify/Unverify an ERC721 collection
  - Set the treasury address
  - Set the creator fee
  - Set the counterparty fee
  - Rescue an ERC721 token stuck
  - Rescue HYPE stuck
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Hyswap project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Hyswap project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] HySwap#rescueERC721 - Unbounded loops when iterating over offers could cause out of gas error

Description

Because the `HySwap` allows for an unlimited number of offers to be created, when the `HySwap` contract iterates over all the offers, the contract may revert when a user tries to interact with the contract.

Vulnerability Details

The `rescueERC721` function loops over all offers to find the offer with the specific NFT address and token ID:

```
for (uint256 i = 0; i < nextSwapOfferId && checked < totalActiveOffers; i++) {
    ...
}
```

Depending on the number of offers that the contract supports, calls to `rescueERC721` may run out of gas, resulting in a denial of service of all external-facing functions.

Impact

Calling the `rescueERC721` function may revert due to running out of gas.

Recommendation

It's recommended that the `rescueERC721` function allows for the specification of an offset and length.

Status

Resolved

[M-02] HySwap#rescueHYPE - Handling offers could fail due to a lack of enough HYPE in the contract

Description

When users create a swap offer, they are required to send native coins (HYPE) for the creator swap fee and creator hype offer and this fund is locked in the contract until the offer is accepted or canceled.

These funds are sent to the buyer or creator when the offers are accepted or cancelled, it means the contract should have sufficient HYPE to make the transactions for accepting or canceling offers successful.

The `rescueHYPE` function is used to rescue HYPE stuck in the contract and it must ensure that there are enough HYPE left in the contract after the rescue.

However, the current function is only checking if there are creator hype offers in the contract after rescue.

Vulnerability Details

The `rescueHYPE` function calculates the total amount of creator hype offers locked and checks whether the contract's HYPE balance after rescue is greater than the locked HYPE.

```
function rescueHYPE(address to, uint256 amount) external onlyOwner nonReentrant {
    ...
      require(address(this).balance - lockedHYPE >= amount, "Insufficient balance");
    ...
}
```

This check only ensures that the contract has sufficient HYPE left for creator hype offers and not for the creator swap fees.

This could allow the contract to not have enough HYPE to cover Creator Hype Offers and Creator Swap Fees.

Furthermore, the transactions for accepting and canceling offers could fail due to a lack of sufficient HYPE in the contract.

Recommendation

It's recommended that the `rescueHYPE` function makes sure that there is enough HYPE left in the contract to cover both Creator Hype Offers and Creator Swap Fees after the rescue.

Status

Resolved

Low

[L-01] HySwap - Use `Ownable2Step` versions rather than `Ownable` versions

Description

`Ownable2StepUpgradeable` and `Ownable2Step` prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

Gas

[G-01] HySwap#setTreasuryAddress, setOfferCreatorSwapFees, setOfferCounterPartySwapFees - Emitting storage variable instead of local variable

Description

The above functions emit a storage variable even though a local variable with the same value is available. While this does not directly affect the correctness of the emitted event, it is inefficient because accessing storage is more expensive in terms of gas compared to accessing a local variable. Emitting the storage variable unnecessarily increases the gas cost of the transaction, which could be avoided by emitting the local variable instead.

```
emit TreasuryAddressUpdated(treasuryAddress) // setTreasuryAddress()
emit CreatorSwapFeesUpdated(creatorSwapFee) // setOfferCreatorSwapFees()
emit CounterPartySwapFeesUpdated() // setOfferCounterPartySwapFees()
```

Recommendation

To optimize gas usage, emit the local variable instead of the storage variable when the local variable holds the same value. This reduces the number of storage reads and improves the overall efficiency of the contract.

Status

Resolved

[G-02] HySwap#rescueERC721 - Cache array length before looping

Description

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

```
for (uint256 j = 0; j < offer.creatorERC721Addresses.length; j++) {
    ...
}

for (uint256 j = 0; j < offer.counterPartyERC721Addresses.length; j++) {
    ...
}
```

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array.

Status

Resolved

QA

[Q-01] HySwap - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Status

Resolved

[Q-02] HySwap - Presence of forge console import

Description

The contract contains an import statement for the Forge console (`console.sol`), which is typically used for debugging purposes during development. While this does not affect the functionality of the contract, leaving such imports in production code is unnecessary and can be considered poor practice. It may also increase the contract's bytecode size, leading to slightly higher deployment costs.

Recommendation

Remove the `console.sol` import from the contract before deploying it to production. This ensures that the contract is clean, optimized, and free of unnecessary development artifacts.

Status

Resolved

Centralisation

The Hyswap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Hyswap project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au